FedNIDS: A Federated Learning Framework for Packet-based Network Intrusion Detection System

QUOC H. NGUYEN*, University of South Florida, USA
SOUMYADEEP HORE*, University of South Florida, USA
ANKIT SHAH, University of South Florida, USA
TRUNG LE, University of South Florida, USA
NATHANIEL D. BASTIAN, United States Military Academy, USA

Network intrusion detection systems (NIDS) play a critical role in discerning between benign and malicious network traffic. Deep neural networks (DNNs), anchored on large and diverse data sets, exhibit promise in enhancing the detection accuracy of NIDS by capturing intricate network traffic patterns. However, safeguarding distributed computer networks against emerging cyber threats is increasingly challenging. Despite the abundance of diverse network data, decentralization persists due to data privacy and security concerns. This confers an asymmetric advantage to adversaries, as distributed networks face the formidable task of securely and efficiently sharing non-independently and identically distributed data to counter cyber-attacks. To address this, we propose the Federated NIDS (*FedNIDS*), a novel two-stage framework that combines the power of federated learning and DNNs. It aims to enhance the detection accuracy of known attacks, robustness and resilience to novel attack patterns, and privacy preservation, using packet-level granular data. In the first stage, a global DNN model is collaboratively trained on distributed data, and the second stage adapts it to novel attack patterns. Our experiments on real-world intrusion data sets demonstrate the effectiveness of *FedNIDS* by achieving an average F1 score of 0.97 across distributed networks and quickly disseminating novel attack information within a few rounds of communication.

CCS Concepts: • **Security and privacy → Network security**; **Intrusion/anomaly detection and malware mitigation**; • **Computing methodologies → Machine learning algorithms**.

Additional Key Words and Phrases: Federated learning, network intrusion detection systems, distributed and private network data, packet-based NIDS, novel attack detection

ACM Reference Format:
Quoc H. Nguyen, Soumyadeep Hore, Ankit Shah, Trung Le, and Nathaniel D. Bastian. 2023. FedNIDS: A Federated Learning Framework for Packet-based Network Intrusion Detection System. *Digit. Threat. Res. Pract.* , (2023), 20 pages. https://doi.org/XXXXXXX.XXXXXXX

*Both authors contributed equally to this research.

Authors' addresses: Quoc H. Nguyen, nguyenq29@usf.edu, University of South Florida, 4202 E. Fowler Avenue, Tampa, Florida, USA, 33620; Soumyadeep Hore, University of South Florida, 4202 E. Fowler Avenue, Tampa, Florida, USA, 33620; Ankit Shah, ankitshah@usf.edu, University of South Florida, 4202 E. Fowler Avenue, Tampa, Florida, USA, 33620; Trung Le, tqle@usf.edu, University of South Florida, 4202 E. Fowler Avenue, Tampa, Florida, USA, 33620; Nathaniel D. Bastian, nathaniel.bastian@westpoint.edu, United States Military Academy, 2101 New South Post Road, West Point, USA, nathaniel.bastian@westpoint.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Association for Computing Machinery.
2576-5337/2023/-ART
https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

The proliferation of digital systems and the increased reliance on interconnected networks have elevated the significance of ensuring the security of computer systems and data. Network intrusion detection systems (NIDS) play a pivotal role in identifying potential threats and attacks within computer networks. These systems employ signature-based [14, 28] and anomaly-based [15, 47] techniques to detect attacks. With advancements in algorithmic knowledge and the availability of high-performance computing resources, deep learning (DL) methods have taken center stage in the development of state-of-the-art NIDS. DL-based NIDS are developed using both supervised [2, 6] and unsupervised [22, 48] training methods. With the assistance of labeled training data containing attack categories, the supervised learning paradigm has demonstrated superior performance in identifying previously known attacks when compared to the unsupervised learning approach. The type of data used for training DL-based NIDS can be categorized into flow-based or packet-based data. While flow level data that provides summaries of network activities by aggregating information from packets is useful for network performance monitoring, the granular and rich information from raw packet level data can be used for real-time detection and immediate response to security incidents [9, 16, 18].

DL-based NIDS rely on the availability of large, consolidated data sets to learn the underlying patterns of benign and malicious network traffic. However, accumulating and consolidating data across different entities in distributed computer networks poses challenges in various situations: (i) Different branches within an organization, such as those in banks, hospitals, or the Department of Defense, often handle sensitive and confidential information that cannot be shared in a central location to train DL models. (ii) Many regions, like the European Union, and industries, such as healthcare under HIPAA regulations, enforce strict data privacy and security regulations that restrict the movement and sharing of data. (iii) Networks can exhibit significant variations in terms of their size and traffic patterns, and aggregating their data to train a centralized model may not necessarily result in superior detection accuracy across different network segments. (iv) Transmitting large volumes of data to a centralized server for DL model training may demand substantial bandwidth and introduce latency. Hence, this approach may not be well-suited for environments constrained by limited bandwidth and low-latency requirements, such as communication in military operations or real-time monitoring and transmission using internet of things (IoT) devices in an industrial setting. Another challenge in NIDS is the delayed identification of new attack patterns, such as zero-day and evasion attacks. This gap becomes particularly significant in a distributed setting where various entities cannot share novel attack data among themselves. This situation necessitates an adaptive mechanism for rapidly disseminating the knowledge from the affected participants. These challenges underline the need for alternative approaches in DL-based NIDS, which can address data privacy concerns, including regulatory constraints, and the variability in types of attacks experienced by the distributed entities while optimizing network intrusion detection.

Federated learning paradigm provides a compelling solution to the aforementioned challenges, empowering the collaborative training of models across multiple clients (distributed networks) without requiring centralization of data. This collaborative paradigm preserves data privacy and security, and involves clients training their local models on their private data. Subsequently, the clients share the model updates in a secure and aggregated manner [11]. We introduce a novel framework called the *Federated Network Intrusion Detection System (FedNIDS)*, which leverages the synergy of federated learning and deep neural networks (DNNs) to enhance the accuracy and robustness of the NIDS against an evolving adversarial environment. *FedNIDS* aims to address the challenges in accurate detection of malicious network activities while upholding data privacy and security by training the DNNs on decentralized and privacy-sensitive packet-based data sources.

The primary contribution of this study is the development of the federated learning framework, *FedNIDS*, designed to operate in a distributed environment, utilizing granular packet-level data. The objective of this framework is to identify known attack patterns and adapt to detect emerging threats. It consists of two essential

stages: federated DNN pre-training and federated novel attack fine-tuning. In the first stage, the framework trains a global DNN-based binary classifier by leveraging decentralized data from participating clients. The second stage involves fine-tuning the pre-trained global model from the first stage with novel attack samples, as observed by the client(s). Our study is the first to address the issue of non-independently and identically distributed (non-IID) data across different clients for packet-based NIDS. We employ publicly available benchmark network intrusion detection data sets for experimentation and evaluation. The results illustrate the robustness and resilience of our *FedNIDS* framework in a distributed environment when encountering various forms of attacks, including known and unknown threats, such as zero-day and adversarially perturbed evasion attacks. These findings collectively contribute to the advancement of NIDS technology, enhancing their ability to identify and mitigate a broad spectrum of network attacks, particularly in situations where network packet data among different clients is non-IID.

The structure of the remainder of the paper is as follows: Section 2 reviews the existing literature on NIDS and federated learning. Section 3 outlines our methodological framework, *FedNIDS*. In Section 4, we delve into numerical experiments. Section 5 presents the results and offers an in-depth analysis. Finally, in Section 6, we draw conclusions from our study and suggest potential avenues for future research.

2 RELATED WORK

We present the related literature review by dividing this section into the following sub-sections: (i) NIDS, (ii) federated learning, and (iii) federated learning in NIDS.

2.1 Network Intrusion Detection Systems (NIDS)

NIDS play a critical role in safeguarding computer networks from unauthorized access and malicious activities. The entire NIDS literature can be broadly categorized into signature-based and anomaly-based NIDS. A signature-based NIDS relies on the matching of seen attack signatures. An anomaly-based NIDS tries to model the expected user behavior to detect any possible deviation from the normal user behavior. With advancements in computational technology and the availability of faster computational resources anomaly-based NIDS have gained more popularity recently. Anomaly-based NIDS rely on DL and statistical learning methods to identify malicious activities [32]. To enhance the effectiveness of NIDS, the integration of DNNs has emerged as a powerful approach. DNNs excel in capturing intricate relationships and complex patterns within network traffic data [20]. Their inherent ability to learn hierarchical features makes them well-suited for discerning subtle deviations in network behavior that may signify malicious activity [49]. Additionally, DNNs can dynamically adapt to evolving attack patterns by fine-tuning and retraining on new data, ensuring persistent accuracy in detecting emerging threats [56]. Their ability to model non-linear relationships proves especially advantageous for capturing intricate attack behaviors that could evade traditional methods [25]. By capitalizing on these strengths, DNNs offer a promising avenue to elevate the accuracy and robustness of anomaly detection within packet-based NIDS.

Depending on the training data required to train such anomaly-based NIDS there exist two prominent approaches in NIDS design: flow-based and packet-based methods. We provide insights into these approaches, highlighting their strengths, weaknesses, and recent advancements. Flow-based NIDS operates by analyzing network traffic at the flow level, where a flow represents a sequence of packets between two network endpoints. These systems focus on extracting features from flow data to identify anomalies and attacks [38, 44, 46]. Packet-based NIDS, on the other hand, inspects individual packets in the network traffic. They analyze packet contents, headers, and payloads to detect malicious patterns [9, 13]. Some of the advantages of using packet-based NIDS over flow-based are as follows: 1) Packet-based NIDS can perform in-depth packet inspection, allowing them to detect attacks with high precision [27]. 2) Flow-based NIDS provides a coarser view of network traffic, which makes it challenging to detect certain types of attack [3]. 3) Packet-based NIDS excel at content-based matching,

making them very effective against known attacks [36]. 4) Flow-based NIDS relies on different Netflow tools (such as Wireshark and CIC flowmeter) for compiling network packet information to obtain features for training, causing a loss of generalizability across different end users. Using raw packet features as training data alleviates the generalizability issue [22]. 5) Packet-based features enable real-time detection since flow-based features are obtained by analyzing the network communication after the flows are completed [16, 21].

2.2 Federated Learning

Federated learning has gained substantial attention as a transformative approach to decentralized machine learning. It enables model training across distributed data sources while preserving privacy and data locality. We provide an overview of key concepts, recent advancements, and notable research papers in the field of federated learning. McMahan et al. (2017) defines federated learning, in their seminal paper "Communication-Efficient Learning of Deep Networks from Decentralized Data", as an ML framework for training models across decentralized edge devices while keeping data local [34]. This approach has evolved from the necessity to address privacy concerns and the increasing prevalence of edge and internet of things (IoT) devices. Federated learning typically consists of three main components: client devices (edge nodes), a central server, and federated learning algorithms. The client devices hold local data and perform local model updates, while the central server coordinates model aggregation and global updates [11]. The federated averaging (FedAvg) algorithm, introduced by McMahan et al. (2017), is a fundamental algorithm that aggregates model updates from client devices to construct a global model. It involves iterative rounds of communication between clients and the central server. To ensure privacy, differential privacy mechanisms have been integrated into the federated learning algorithm. For example, techniques like DP-SGD [1] and secure aggregation [11] protect user data from exposure during model updates. Federated learning often assumes that client data distributions are independent and identically distributed (IID). Research has focused on addressing the challenges posed by non-IID data, such as skewed distributions and non-uniform data access patterns among others [30]. Li. et al. (2020) modified the FedAvg algorithm to address non-IID data challenges by extending the standard federated optimization objective by adding a proximal/regularization term to the loss function [31]. The algorithm is popularly known as FedProx [31], which encourages model updates from clients to be close to their previous models by effectively penalizing larger updates. The scope of federated learning's applications has transcended traditional boundaries, extending to domains like healthcare [55] and the IoT [53], accentuating its relevance for NIDS as well [29, 40].

2.3 Federated Learning in NIDS

Next, we discuss the use of federated learning in NIDS. In Table 1 we highlight some of the most recent and notable works employing federated learning in the NIDS domain. We observe that a major part of the literature focuses on building a federated learning framework for flow-based NIDS [4, 5, 7, 12, 23, 39, 45]. One of the major challenges in implementing federated learning for NIDS is data dimensionality as extracting flow-based features from different source data sets results in an inconsistent set of features across different clients. Packet-based implementation of the federated learning framework alleviates that problem since raw packet data features can be used directly for training, which remain consistent across the clients, irrespective of the source data sets. There exist very few works in the literature that employ a packet-based NIDS with the federated learning framework. Willeke et al. (2023) [52] use raw packet features extracted from the packet capture (PCAP) files, while Nguyen et al. (2019) [37] consider some extracted features like packet direction, port type, and transmission control protocol (TCP) flags, among others. Additionally, the latter also considers a sequence of packets between the security gateway and IoT devices for the detection task. Another important challenge in implementing federated learning in NIDS is the model divergence under non-IID data scenarios. The FedAvg algorithm, used in [52], performs well under IID assumptions. However, distributed clients in the NIDS setting experience non-IID data. The FedAvg

Table 1. A brief literature review of federated learning in NIDS

Paper Name	Authors	Year	Algorithm	Global/ Local Model	Data Representation	Data Set(s)
Fed-ANIDS: Federated learning for anomaly-based NIDS [23]	Idrissi et al.	2023	FedProx	Autoencoders	Flow-based	USTC-TFC2016, CICIDS2017, CICIDS2018
Data-efficient, federated learning for raw network traffic [52]	Willeke et al.	2023	FedAvg	1dCNN, FcNN	Packet-based	UNSW-NB15, CICIDS-2017
GowFed A novel federated network intrusion detection system [7]	Belenguer et al.	2023	Fedavg with Gower distance	DNN	Flow-based	TON_IOT
Enhancing privacy-preserving intrusion detection through federated learning [4]	Alazab et al.	2023	FedAvg	DNN	Flow-based	NSL-KDD
Collaborative IDS for SDVN: A fairness federated deep learning approach [12]	Cui et. al	2023	FedAvg with two-stage gradient optimization	1dCNN	Flow-based	KDD99, NSL-KDD
FLUIDS: Federated Learning with semi-supervised approach for IDS [5]	Aouedi et al.	2022	FedAvg	Autoencoders, DNN	Flow-based	UNSW-NB15
Towards asynchronous federated learning-based threat detection: A DC-adam approach [45]	Tian et al.	2021	DC-adam	Denoising Autoencoders	Flow-based	CICIDS-2017, IoT-23, MNIST
Internet of things intrusion detection: centralized, on-device, or federated learning [39]	Rahman et al.	2020	FedAvg	DNN	Flow-based	NSL-KDD
DIOT: A federated self-learning anomaly detection system for IoT [37]	Nguyen et al.	2019	FedAvg	GRU	Packet-based	Mirai

algorithm is known to have underwhelming results in the presence of non-IID data across the different clients. The DC-adam algorithm addresses this issue by using Taylor expansion to stabilize the loss function when minimizing it along with some warm-up procedures for parameter initialization [45]. Belenguer et al. in the GowFed paper address the model divergence issue by introducing a Gower dissimilarity [17] to limit the extent of the model updates. The FedProx algorithm [31] also performs well, in general, in the presence of non-IID data. Recently, some works have also explored poisoning attacks on federated learning-based NIDS [35, 54]. However, evaluating the resilience of federated learning-based NIDS classifier systems under novel attacks such as zero-day and evasion attacks remains unexplored.

To the best of our knowledge, there is a lack of existing literature on the development of a federated learning framework for NIDS using raw packet data in a non-IID data environment. There is also a notable absence of studies that have assessed and enhanced the robustness of federated learning-based NIDS in the face of emerging and unfamiliar attack scenarios. In this study, we aim to address these gaps by developing a federated learning framework for NIDS that is capable of recognizing known attacks and can be fine-tuned to quickly adapt to identify novel attacks across different clients.

3 METHODOLOGY

3.1 Preliminaries

3.1.1 Federated Learning: A Mathematical Overview. Federated Learning in the client-server paradigm involves three main parties: 1) a multitude of silos (e.g., clients) denoted as $S = \{s_i\}_{i=1}^{N}$, where each s_i owns private data sets D_{s_i}; 2) a model owner, O, who initiates the process with a randomly initialized global model, represented by its parameters, w_G, and oversees its training; and 3) a secure aggregator *Agg* (e.g., a trusted server), positioned between O and S. Notably, O is oblivious to the specifics of D_{s_k} due to the implementation of a secure aggregation protocol by *Agg* [11]. Typically, stochastic gradient descent (SGD) is used by the silos to train their models (FedAvg [34]). Preceding the commencement of federated learning, w_G is initialized by O. During the t-th aggregation round:

(1) *Agg* transmits w_G^t to a subset of silos $S_{sub} = \{s_i\}_{i=1}^{K}$, where $K \leq N$.
(2) Each silo s_i updates its local model from $w_{s_i}^{t-1}$ to $w_{s_i}^t = w_G^t$. Subsequently, s_i retrains $w_{s_i}^t$ through a predetermined number of local passes over its private data set D_{s_i} and sends the retrained local model $w_{s_i}^t$ back to *Agg*.
(3) *Agg* aggregates the retrained local models to compute a new global model w_G^{t+1}.

The above process continues iteratively, until the global model w_G achieves the desired performance level or another predefined stopping criterion is met. The objective is to improve the global model's accuracy over successive rounds, while respecting privacy and security constraints.

3.1.2 Non-IID Data Issue in Federated Learning. A significant challenge within the realm of federated learning arises due to the presence of non-IID data across different silos [24, 30]. Non-IID data refers to data that does not share the same statistical characteristics among various silos. This non-IID nature of data can significantly impact the accuracy of the federated learning algorithm. The underlying reason lies in the distinctiveness between the distribution of each local data set within different silos and the global distribution. Consequently, the objectives of the local models within each silo can be misaligned with the overarching goal of achieving global optimality. This misalignment leads to a *drift* in the local updates [26, 50, 51].

To simplify, during the local training phase within each silo, the models strive to improve their respective local objectives. However, these local objectives may lead the models to local optima that differ considerably from the desired global optima. Moreover, when substantial local updates occur, such as an extensive number of local training epochs [31], the average model derived from combining these local models can also deviate significantly from the global optima. As a result, the final global model's accuracy is notably compromised, particularly when compared to scenarios where data is IID. This challenge is visually illustrated in Fig. 1, which vividly captures the hurdle faced by FedAvg when dealing with non-IID data scenarios involving silos. To elaborate further, consider

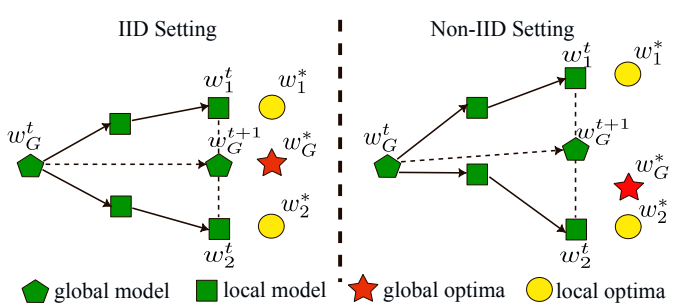

Fig. 1. Example of a drift under the non-IID setting.

the scenario where the global optimum w_G^* is close to the local optima w_1^* and w_2^* under the IID setting. This results in the averaged model w^{t+1} being close to the global optimum. However, in non-IID settings, when w_G^* is distant from w_1^*, the averaged model w^{t+1} can deviate significantly from the global optimum. Effectively tackling this challenge requires designing robust algorithms tailored to handle non-IID data present across different silos.

3.1.3 Problem Statement. Consider N clients or silos denoted as $S = \{s_i\}_{i=1}^N$, each possessing a local data set D_{s_i} containing network traffic samples. The objective is to develop a global classification model that can accurately identify malicious samples within the network traffic data from these distributed data sets, while upholding data privacy and security. We pose the problem as follows: Given the distributed data sets D_{s_i}, develop a federated learning-based DNN model that optimizes a global objective function while respecting the privacy constraints of individual clients. Mathematically, we aim to minimize the global objective function $\mathcal{L}(w)$, which is defined as:

$$\arg\min_w \mathcal{L}(w) = \sum_{i=1}^{N} \frac{|D_{s_i}|}{|D|} \mathcal{L}_i(w)$$

In the above equations:

- $|D_{s_i}|$ represents the size (number of samples) of the local data set for a particular silo s_i.
- $|D|$ represents the total size (number of samples) of the entire data set, which comprises the combined data from all silos.
- $\mathcal{L}_i(w)$ is the local objective function for silo s_i, measuring the goodness of fit or the loss of the global model w on the data specific to silo s_i. It is defined as $\mathcal{L}_i(w) = \mathbb{E}_{x;y\sim D_{s_i}}[\ell_i(w;x;y)]$.

The optimization problem, $\arg\min_w \mathcal{L}(w)$, seeks to find the global model parameters w that minimize the objective function $\mathcal{L}(w)$ ensuring effective performance across all data sources while respecting privacy constraints in federated learning. This is achieved by aggregating local losses $\mathcal{L}_i(w)$ from each source, weighted by their data contributions, to formulate $\mathcal{L}(w)$. Minimizing $\mathcal{L}(w)$ aims to create a model that balances performance across all sources, promoting effectiveness on the combined data from multiple parties, the core objective in distributed data scenarios. Our research aims to develop a federated learning-based DNN model that can effectively detect attacks across different silos and efficiently adapt to novel attack patterns when new data becomes available.

3.2 Federated Learning Framework for Packet-based Network Intrusion Detection System (*FedNIDS*)

To attain the aforementioned research objective, we propose a generalized federated learning framework, named *FedNIDS*, to enhance both the robustness and the performance of federated models when learning from decentralized data with statistical heterogeneity in NIDS. Our framework comprises two stages: a federated DNN pre-training stage and a federated novel attack fine-tuning stage, as shown in Fig. 2 and Alg. 1. During the federated DNN pre-training stage, the model exploits knowledge from decentralized data by pre-training with pre-processing procedure in a distributed setting. In the federated novel attack fine-tuning stage, the knowledge is transferred from the previous stage to the target task by fine-tuning the federated models with novel attack samples.

3.2.1 Federated DNNs Pre-training. The first stage of our proposed *FedNIDS* framework focuses on enhancing the global model's initialization through federated DNN pre-training. The goal is to develop robust initialization of local DNNs at each silo that can effectively capture relevant features from their respective data sets. This stage reduces the need for extensive communication rounds between the clients and the server, thereby enhancing the efficiency of the federated learning process.

The pre-training process occurs in a distributed setting and involves the following steps, repeated for a maximum of T communication rounds:

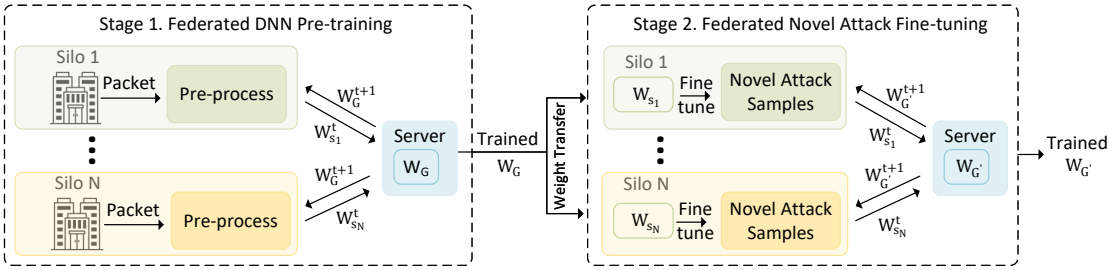

Fig. 2. Overview of the federated learning framework. In the pre-training stage (left), the packet data is processed to learn representations in each silo. The pre-training process consists of three steps and ends when it reaches the maximum communication rounds T. At round t, (1) Each silo $S_i (i \in \{1, ..., N\})$ trains its local DNN with local data; (2) Silo s_i uploads the weights $w_{s_i}^t$ to the central server; (3) The server produces a global DNN model with weights $w_{s_i}^{t+1}$ via model weights averaging and broadcasts the global model back to each local silo. In the fine-tuning stage (right), the final pretrained global W_G from the first stage is used to initialize each local DNN. The W_G is then fine-tuned with new attack samples. End-to-end federated fine-tuning is performed on the novel attack samples in each local silo to get the desired model $W_{G'}$.

(1) Pre-processing: Numerical features are extracted from the raw packet capture (pcap) data files. The pre-processing step is kept simple and generalizable across different organizations. As discussed earlier, flow-based features raise the issue of inconsistent feature sets across organizations and hinders generalizability. Hence, we use the raw pcap files to extract numerical features. The data set creation tool is adopted from [21]. A comprehensive guide to extracting and labeling the pcap files are as follows:
   - Use Scapy [10] and dpkt [43] to parse different header and segment information from the raw PCAP file.
   - Use the 6-tuple information containing source internet protocol (IP) address, destination IP address, source port, destination port, protocol, and epoch time to differentiate between benign and malicious packets.
   - Extract only forward packets resulting in a data set of unidirectional communication, i.e. only extract packets that are being sent from identified source IP addresses excluding any response from the server.
   - To alleviate inherent bias in training ML models, we remove some header information (ethernet header, source and destination IP address, source and destination port information).
   - Set a maximum feature length for all extracted packets. However, the number of bytes in a packet may vary greatly. To counter this zero-padding is used that helps in maintaining a standard structure of the data.
   - Convert raw packet information from hexadecimal to decimal. Each byte value contains a combination of two hexadecimal numbers, ranging from 00 to FF. The hexadecimal numbers are converted to decimal numbers between $0 - 255$ and subsequently normalized for efficient machine computation.
(2) Each silo s_i (where $i \in \{1, \ldots, K\}$) trains its local DNN-based classifier with its own preprocessed data for E local epochs.
(3) Silo s_i uploads the weights $w_{s_i}^t$ of its DNN to the central server.
(4) The central server aggregates the uploaded weights using a model weights averaging technique and produces an updated global DNN model with weights w^{t+1}.
(5) The server broadcasts the updated global model to each local silo s_i.

In addition to the pre-training stage, our *FedNIDS* framework incorporates the *FedProx* technique [45] to enhance the local objective function based on the FedAvg algorithm. This technique introduces an $L2$ regularization

Algorithm 1 FedNIDS

Input: Number of communication rounds T, number of local epochs E, number of silos N, local dataset D_{s_i}, learning rate η

Output: Global models $w_G, w_{G'}$

Server executes:

Initialize w_G^0;
for each round $t = 0, 1, ..., T-1$ **do**
  Sample a set of clients S^t;
  $n \leftarrow \Sigma_{i \in S^t} |D_{s_i}|$;
  for each silo $s_i \in S^t$ in parallel **do**
    Send the global model w_G^t to silo s_i;
    $\Delta w_{s_i}^t \leftarrow$ **LocalTraining**(s_i, w_G^t);
▷ *Stage 1: Federated DNN Pre-training*
  $w_G^{t+1} \leftarrow \Sigma_{i=1}^{N} \frac{|D_{s_i}|}{n} \Delta w_{s_i}^t$;
▷ *Stage 2: Federated Novel Attack Fine-tuning*
  $w_{G'}^{t+1} \leftarrow \Sigma_{i=1}^{N} \frac{|D_{s_i}|}{n} \Delta w_{s_i}^t$;

Client executes:

$\mathcal{L}(w; \mathbf{b}) = \sum_{(x,y) \in \mathbf{b}} \ell(w; x; y) + \frac{\mu}{2} ||w - w_t||$;
LocalTraining(s_i, w_G^t);
▷ *Stage 1: Federated DNN Pre-training*
Sample batches $\mathbf{b} = \{x, y\}$ from local data D_{s_i};
$\mathbf{b}' \leftarrow$ Pre-process($\mathbf{b}$);
$w_{s_i}^t \leftarrow w_G^t$;
for each local epoch $k = 1, ..., E$ **do**
  for each batch $\mathbf{b}'$ **do**
    $w_{s_i}^t \leftarrow w_{s_i}^t - \eta \nabla \ell(w_{s_i}^t; \mathbf{b}')$;
$\Delta w_{s_i}^t \leftarrow w_G^t - w_{s_i}^t$;
▷ *Stage 2: Federated Novel Attack Fine-tuning*
for each local epoch $k = 1, ..., E$ **do**
  for each batch $\mathbf{b}'$ **do** ▷ Updated batch $\mathbf{b}'$ with novel attack samples
    $w_{s_i}^t \leftarrow w_{s_i}^t - \eta \nabla \ell(w_{s_i}^t; \mathbf{b}')$;
$\Delta w_{s_i}^t \leftarrow w_{G'}^t - w_{s_i}^t$;
return $\Delta w_{s_i}^t$ to the Server;

term in the local objective function to limit the distance between the local model and the global model. This regularization term effectively controls the size of local updates, preventing the local models from diverging too far from the global optima. A hyper-parameter μ is introduced to adjust the weight of the $L2$ regularization term. While the modification to FedAvg is lightweight and easy to implement, it introduces some additional computation overhead. However, it does not introduce extra communication overhead. This approach requires fine-tuning of μ to achieve superior accuracy. If μ is too small, the regularization term might have minimal impact. Conversely, if μ is too large, the local updates become very small, potentially leading to slower convergence speed.

This stage concludes when the maximum communication rounds T are reached. The global model w_{T+1} obtained at the end of the pre-training stage serves as a robust initialization for the subsequent novel attack fine-tuning stage. The combination of pre-processing and federated DNN pre-training enhances the quality of the learned features and the global model's ability to capture important characteristics of network traffic data, improving the overall effectiveness of the *FedNIDS* framework.

3.2.2 Federated Novel Attack Fine-tuning. In the second stage of our FedNIDS framework, we perform federated novel attack fine-tuning by utilizing the knowledge gained during the pre-training stage to the target task of network intrusion detection. This stage aims to enhance the global model's ability to learn new and emerging attack patterns by fine-tuning it using novel attack samples experienced at some silos.

The fine-tuning process occurs as follows:

(1) The trained global model w_{T+1} from the pre-training stage is used to initialize each local DNN at the respective silos.
(2) Each silo s_i continues to contribute to the learning of the global model with their own local data sets D_{s_i}, which include newly experienced novel attack samples.
(3) The fine-tuning involves updating the DNN's parameters, w, using an optimization algorithm such as SGD to minimize the loss on the novel attack samples.

This federated novel attack fine-tuning stage effectively adapts the trained global model to the specific characteristics of each silo's data set, enabling the model to detect new attack patterns that may arise in the future. The combination of the two stages within the *FedNIDS* framework aims to enhance the model's accuracy, robustness, and generalization capabilities for network intrusion detection.

4 EXPERIMENTS

In this section, we outline the numerical experiments performed to evaluate our *FedNIDS* methodology. We first discuss the experimental data followed by the description related to the non-IID data silo creation. Subsequently, we discuss the hyperparameter configurations employed for both *FedNIDS* and a comparative algorithm, FedAvg [52], recently utilized for NIDS in the literature. The experimentation was carried out employing a computing system featuring the NVIDIA GeForce RTX 3070 graphics processing unit with 12GB of video memory and a substantial 128 GB of DDR4 RAM operating at a speed of 3200MHz. We built a simulator using Python to replicate the federated learning process. The total available computational resources were equally divided amongst the silos.

4.1 Data Description

A federated learning framework in the context of NIDS offers significant advantages in preserving both data privacy and security, all the while maintaining data ownership and control. Nevertheless, obtaining real-world organizational traffic data for research is a daunting task for the same reasons. Hence, researchers turn to publicly available benchmark data sets as a practical substitute. In this study, we use raw PCAP files from two publicly available intrusion detection data sets, namely, CICIDS-2017 [41] and CICIDS-2018 [42] to extract benign and attack communications. We select CICIDS data sets over some popular yet older datasets like NSL-KDD and KDD-CUP due to the following reasons: (i) they provide a more pragmatic representation of modern network traffic [19] and (ii) they provide easy accessibility to the raw PCAP files, thereby reducing the dependency on flow-level data.

These data sets are accumulations of network activities recorded over several days. A brief description of the PCAP files, in terms of their size and contents, is provided in Table 2. We follow the process in [21] to obtain the packet information from the PCAP files. Due to the scarcity of Botnet attack packets, we exclude this attack type from the initial phase of the experimentation. However, we introduce Botnet attack packets extracted from the CICIDS-2017 data set to represent a zero-day attack to evaluate the resilience of our framework. As discussed

Table 2. Description of CICIDS-2017 and CICIDS-2018 PCAP files

CICIDS-2017			CICIDS-2018		
Day/Date	**File Size**	**Activity**	**Day/Date**	**File Size**	**Activity**
Monday/ July 3,2017	10 GB	Benign	Wednesday/ Feb 14,2018	147 GB	SSH/FTP Patator, and Benign
Tuesday/ July 4,2017	10 GB	Brute Force, and Benign	Thursday/ Feb 15, 2018	57.8 GB	DoS Goldeneye, DoS Slowloris, and Benign
Wednesday/ July 5,2017	12 GB	DoS, and Benign	Friday/ Feb 16, 2018	460 GB	DoS Slowhttptest, DoS Hulk, and Benign
Thursday/ July 6,2017	7.7 GB	Web Attack, Infiltration, and Benign	Wednesday/ Feb 21, 2018	97.5 GB	DDoS, and Benign
Friday/ July 4,2017	8.2 GB	Botnet, Port Scan, DDoS, and Benign	Thursday/ Feb 22,2018	110 GB	Web Attack, and Benign
-	-	-	Friday/ Feb 23, 2018	65.9 GB	Web Attack, and Benign
-	-	-	Wednesday/ Feb 28, 2018	73.1 GB	Infiltration, and Benign

earlier, we only consider unidirectional forward packets for the experimentation. The feature set consists of 1525 payload bytes for each sample in our data set. The hex payload values in these features are converted to decimal values between $0 - 255$, and later normalized to values between $0 - 1$.

To evaluate the robustness of the *FedNIDS* framework, we also subject it to adversarial examples generated by modifying different attack packets encountered in the CICIDS data sets. We created two different toolchains to generate these evasion attacks with both constrained and unconstrained perturbations. Toolchain 1 was developed using a DRL approach from literature [21], which makes subtle perturbations such that the functionality and maliciousness of these packets remain intact (constrained). Table 3 shows the features that we considered for perturbations using Toolchain 1 and their resulting impact on other features. Toolchain 2 was developed using a heuristic approach, in which we made perturbations by making a random selection of features (unconstrained). We then methodically increased the feature values by a constant value as in [16]. We considered a white-box setting for the generation of the adversarial samples to test the worst-case scenario for the NIDS. In this setup, we considered a DNN model with the same architecture as used by the global model in the federated learning framework. The samples that evaded the DNN model were collected and stored for each of these toolchains.

4.2 Experiment Setup

Next, we discuss the segregation of the data into four different silos to evaluate the effectiveness of the *FedNIDS* framework. In the FL-NIDS literature, most of the research studies focus on creating a global model with the IID assumption. In a realistic scenario, where we want to create a global NIDS with data coming from different organizations, often it is very difficult to adhere to the IID assumption. To counter this we partition the extracted data based on the shards partition used in [34], a popular method for segregating data into non-IID

Table 3. Description of perturbations used to generate adversarial examples

Perturbation	Description	Impact
Modifying fragmentation	Fragmentation bytes can be modified from do not fragment to do fragment/more fragment or vice versa.	Directly impacts fragmentation bytes (byte numbers 7, 8 of the IP header), indirectly impacts TTL value (byte number 9 of the IP header), and IP checksum (byte numbers 11, 12 of the IP header).
Modifying time to Live	Increasing/decreasing the TTL value by a small integer between $1 - 255$.	Directly impacts the TTL byte (byte number 9 of the IP header), and indirectly impacts IP checksum (byte numbers 11, 12 of the IP header).
Modifying window size	Increasing/decreasing the window size value by a small integer between $1 - 65335$.	Directly impacts window size bytes (byte number 15, 16 of the TCP header), and indirectly impacts TCP checksum (byte numbers 17, 18 of the TCP header).
Adding/modifying maximum segment size	Only limited to SYN and SYN-ACK packets. The MSS value is limited between $0 - 65335$.	Directly impacts TCP options (bytes after 20 of the TCP header), and indirectly impacts TCP checksum (byte numbers 17, 18 of the TCP header).
Adding/modifying window scale value	Only limited to SYN and SYN-ACK packets. The MSS value is limited between $0 - 14$.	Directly impacts TCP options (bytes after 20 of the TCP header), and indirectly impacts TCP checksum (byte numbers 17, 18 of the TCP header).
Adding segment information	Add dead bytes/most commonly occurring segment information.	Directly impacts the TCP segment, and indirectly impacts TCP checksum (byte numbers 17, 18 of the TCP header).

partitions/shards. There exist multiple strategies for shard partitioning, namely, range-based, hash-based, key-based, and round-robin. In this research, we employ hash-based shard partitioning, which distributes data across shards based on a hash function applied to a specific attribute. The selected attribute is the label of the packet data. We created silos based on the selected partitioning strategy. Table 4 shows the total number of malicious and benign forward packets used for training and testing at the four silos for experimentation. Notably, Silo 1 experiences an attack, *Port Scan*, which no other silos encounter. The silos are carefully crafted to retain the non-IID nature of data across them and to show the attack detection efficacy of our framework without explicit data sharing. As discussed in the previous sections, having non-IID data across different silos poses a major challenge to the traditional federated learning frameworks, which our methodology aims to overcome.

To exhibit the non-IID nature of the partitioned silos we perform pairwise hypothesis testing. Particularly, to assess whether two samples have the same underlying distribution, we perform the K-S test [8]. We also perform the Mann-Whitney rank sum test [33] to compare two independent samples to determine if they have different distributions, or if one tends to have larger values than the other. Table 5 exhibits the pairwise K-S test and rank sum test results amongst different silos of data. We perform the tests on a small subset of 500 random samples drawn from each silo of packet data that has some payload information. Each sample from each silo is compared against all other samples from all other silos. The numbers reported in Table 5 are the percentage of times the

Table 4. Data description for different silos

Silo	Packet Type	Source Dataset	Number of Fwd Packets	
			Train	Test
Silo 1	Port Scan, DoS, DDoS, and Benign	CICIDS-2017	133906	33476
Silo 2	Brute Force, Infiltration, Web Attack, and Benign	CICIDS-2017	110585	30146
Silo 3	DoS, Brute Force, and Benign	CICIDS-2018	102947	25736
Silo 4	DDoS, Infiltration, Web Attack, and Benign	CICIDS-2018	102324	25581

K-S test/the rank sum test result indicated towards two samples being different. From Table 4 and Table 5, we can determine that the data partitions we crafted are of non-IID nature.

To demonstrate the effectiveness of the federated learning framework, we consider a setup with four clients, denoted as $N = 4$ silos, actively participating in training the global model during each communication round. The choice of hyperparameters plays a crucial role in achieving optimal performance. We carefully selected the following parameter values based on extensive experimentation: number of rounds ($T = 5$) governs the frequency of communication rounds between clients and the central server, allowing for iterative updates; number of local epochs ($E = 5$) dictates the number of times each client refines its local model on its respective data set, influencing the granularity of updates; a batch size of 32 samples ($b = 32$) determines the number of data samples processed together in each iteration, balancing parallelization and memory constraints; learning rate ($\eta = 0.01$) regulates the step size of parameter updates, influencing the convergence speed; and the regularization parameter ($\mu = 0.005$) helps mitigate overfitting by adding a penalty term based on the model weights. These values were meticulously chosen to strike a balance between convergence speed, model generalization, and available computational resources. In addition to hyperparameters, we tailored the architecture of our DNN to best suit our objectives. The *input_size* was set to 1525, reflecting the number of features in the data set and defining the input data's dimensionality. The network architecture encompassed four hidden layers with sizes of 512, 256, 128, and 64 units respectively, enabling the model to progressively learn abstract features from the input data. The activation function was rectified linear unit (ReLU). The *num_classes* was set to 2, signifying the model's task of predicting between two output classes: benign or malicious network traffic. Consistency in architecture and hyperparameter values was maintained across all algorithms considered for comparison. Please note that to foster reproducibility and contribute to the cybersecurity community, we intend to make the source code and data sets available.

5 RESULTS

In this section, we present the experimental results and conduct an analysis of them. First, we delve into the performance of the global *FedNIDS* model against the various types of attacks observed at different silos. Next, we assess the framework's adaptability to novel attacks, encompassing previously unseen attack types and adversarially perturbed evasion attacks.

Table 5. Percentage of non-IID samples between different silos

Silo	Silo 1		Silo 2		Silo 3		Silo 4	
Statistical Tests	Rank Sum Test	K-S Test	Rank Sum Test	K-S Test	Rank Sum Test	K-S Test	Rank Sum Test	K-S Test
Silo 1	-	-	65.02	90.80	90.52	91.54	93.62	94.45
Silo 2	65.02	90.80	-	-	90.22	98.57	93.20	97.84
Silo 3	90.52	91.54	90.22	98.57	-	-	87.63	88.14
Silo 4	93.62	94.45	93.20	97.84	87.63	88.14	-	-

Table 6. Performance comparison of FedNIDS with other approaches

Silo	Class	FedAvg			Centralized Approach			FedNIDS		
		Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
Silo 1	Attack	1.00	0.97	0.98	0.99	0.98	0.99	**1.00**	**0.99**	**0.99**
	Benign	0.95	1.00	0.98	0.97	0.99	0.98	1.00	1.00	1.00
Silo 2	Attack	1.00	0.96	0.98	0.99	1.00	0.99	**1.00**	**0.99**	**0.99**
	Benign	0.95	1.00	0.97	0.99	1.00	0.99	0.96	0.93	0.95
Silo 3	Attack	1.00	0.68	0.80	0.99	0.90	0.95	**1.00**	**0.95**	**0.96**
	Benign	0.75	1.00	0.86	0.91	0.99	0.95	0.97	1.00	0.96
Silo 4	Attack	1.00	0.67	0.80	0.99	0.90	0.94	**1.00**	**0.96**	**0.94**
	Benign	0.75	1.00	0.86	0.90	0.99	0.94	0.98	0.97	0.95

5.1 Performance of the Global FedNIDS Model

We evaluate the performance of the trained global *FedNIDS* model at each silo and compare it to the FedAvg algorithm [52] and a centralized approach, where the same DNN model is trained using the combined data from all four silos. In the centralized approach, we train the model by consolidating the training sets from each of the silos and evaluating it on the same testing data samples as the other approaches. Table 6 presents precision, recall, and F1 score metrics for both attack and benign classes. We report the performance of each approach in accurately classifying attack and benign samples for each silo. Notably, the global *FedNIDS* model exhibits the strongest performance across all the silos and among all the methods, achieving the highest precision and recall values for attack samples.

For Silos 1 and 2, *FedNIDS* demonstrates remarkable precision, recall, and F1 score values for both attack and benign classes. Specifically, for the attack class, the model achieves precision, recall, and F1 score values greater than or equal to 0.99, indicating its ability to accurately identify attack samples with minimal false positives or false negatives. Similarly, for the benign class, the model exhibits high precision and recall values, resulting in F1 scores of 1.00 and 0.95, respectively. This showcases the model's ability to maintain accurate classification of benign samples, while effectively detecting attacks.

In Silos 3 and 4, where attack patterns are more complex and diverse, the global *FedNIDS* model maintains strong performance. For the attack class in Silos 3 and 4, the model achieves precision values of 1.00 and recall values greater than or equal to 0.95, with F1 scores at or above 0.95. This reflects the model's ability to effectively classify various attack samples. Furthermore, in Silo 3, the model demonstrates high precision (0.97) and recall (1.00) for the benign class, resulting in an F1 score of 0.96. In Silo 4, the model achieves a precision of 0.98 and a recall of 0.97 for the benign class, yielding an F1 score of 0.95. These outcomes underline the robustness and adaptability of the global *FedNIDS* model to different types of samples in the silos' data sets.

An intriguing observation is that *FedNIDS* attains a similar level of performance as the centralized approach. This is an important finding as it highlights the potential of a federated learning framework to achieve detection outcomes on par with traditional centralized models. Additionally, this finding underscores the feasibility of implementing *FedNIDS* in real-world decentralized network environments. By distributing the training process and leveraging local data, *FedNIDS* effectively adapts to the distinct threat landscapes of various silos, ultimately contributing to an accurate and responsive NIDS. Figure 3 (a) shows the loss values at all the silos converging by the end of the fourth round.

5.2 Adaptation to New Threats

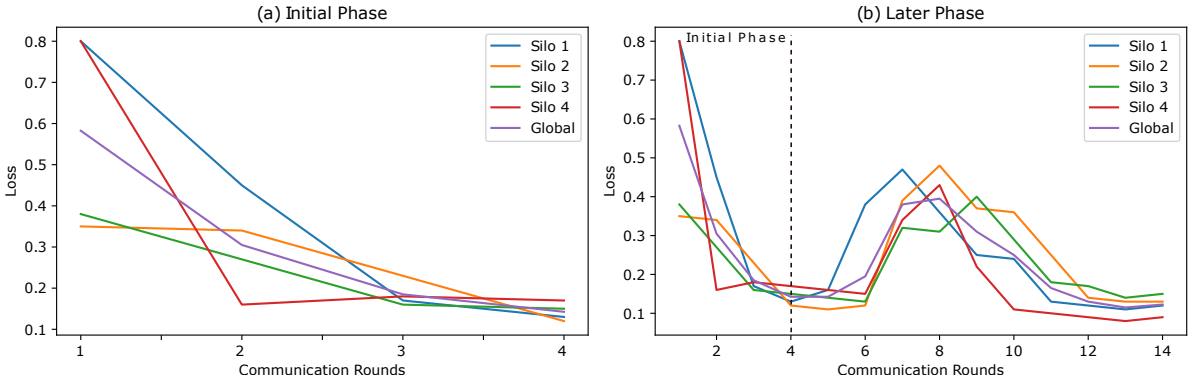

Fig. 3. FedNIDS adaptation to emerging threats.

Network security is a dynamic field, demanding adaptive and robust solutions to counter emerging threats. In this experiment, we evaluate the capability of *FedNIDS* to adapt to new, previously unseen attacks. To perform this evaluation, we introduce held-out botnet attack samples, which had not been encountered by any of the silos previously. Silo 1 is exposed to these novel attack samples. It is important to note that, at the time of this experiment, all silos were employing the trained global model, whose performance had been detailed in the previous experiment after four rounds of communication (refer to Figure 3 (a)).

This experiment aims to demonstrate the efficacy of the second stage of the *FedNIDS* framework. Figure 3 (b) illustrates the subsequent rounds of updates between the clients and the server (as outlined in Algorithm 1), starting from the moment Silo 1 encountered the botnet attack samples. Initially, it is evident that the trained global model struggles with detecting the botnet attack pattern from Silo 1. This challenge is reflected in the higher loss values, indicating suboptimal performance during the early rounds. Nevertheless, as the training progresses with additional rounds, the global model gradually learns to recognize the botnet attack pattern through the weight updates originating from the Silo 1 model.

After approximately four additional rounds, the model exhibits a significant reduction in loss across all silos, signifying a successful adaptation to the detection of botnet attack samples. The global model curve, which represents the average performance of all silos, illustrates the collective learning process. We conducted multiple trials involving various previously unseen attacks from the CICIDS data sets and consistently observed that *FedNIDS* could adapt to new threats within approximately four rounds, on average. This experiment effectively demonstrates that the *FedNIDS* framework can adeptly respond to new threats, providing an efficient solution for sharing knowledge (rather than raw data) about a new attack experienced at one silo with all the participants.

For a real-world implementation, it is crucial to take into account the frequency of communication rounds between the clients and the central server. Depending on the unique requirements and threat landscape at each silo, the update intervals may differ. Some silos might necessitate more frequent updates, perhaps on a daily, weekly, or monthly basis, to guarantee timely adaptation to emerging threats. This adaptive scheduling of communication rounds can be customized to suit the specific needs of individual silos, enabling a finely tuned response to evolving security challenges.

5.3 Evaluating the Resilience of FedNIDS against Adversarial Attacks

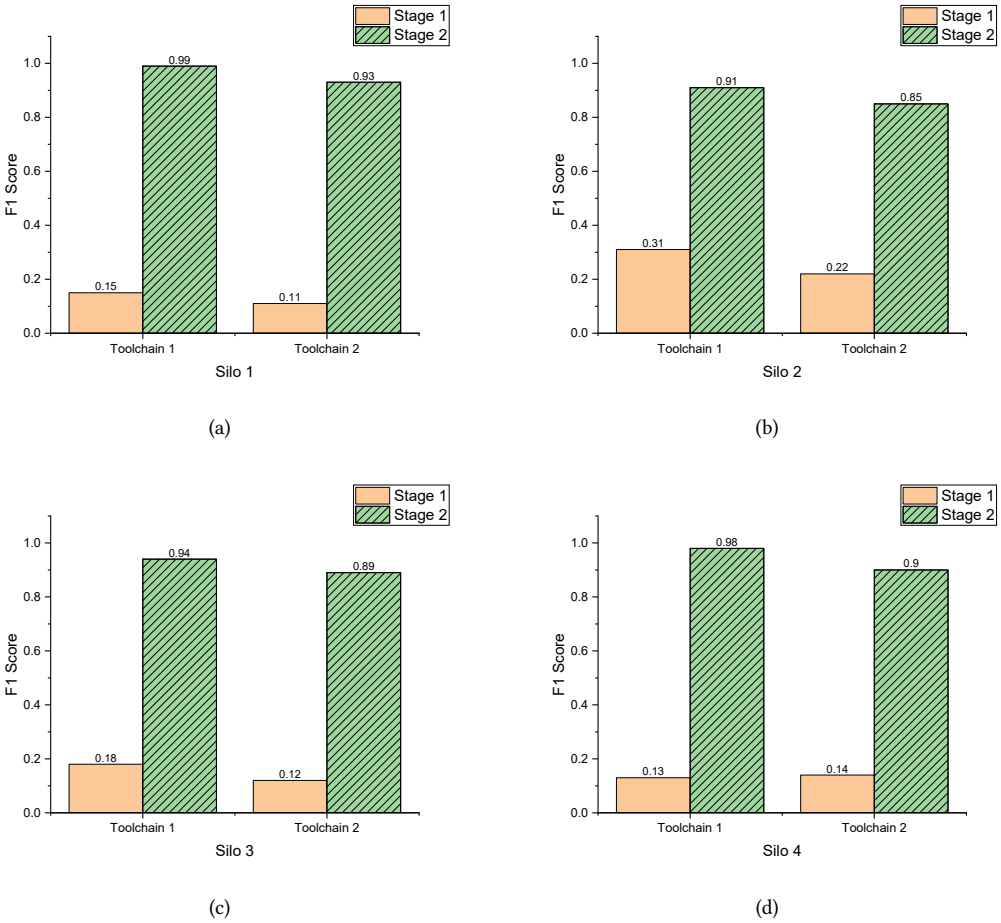

Fig. 4. The performance of FedNIDS against adversarial attacks after fine-tuning.

The next experiment is designed to provide a comprehensive assessment of the resilience of *FedNIDS* against adversarial samples at two critical stages: immediately after the initial training (Stage 1) and following fine-tuning with additional adversarial data (Stage 2). The results from this experiment offer crucial insights into how *FedNIDS*

adapts to adversarial data and whether the fine-tuning process, aided by an increased quantity of adversarial samples, enhances its effectiveness.

The evaluation process was conducted as follows: We employed adversarial data sets generated by the two toolchains, as elaborated in the previous section (4), for this assessment. These data sets were created to mimic potential real-world evasion attacks. During Stage 2, we divided the total number of samples that successfully evaded the DNN model using these toolchains into testing and training sets. In Stage 1, we assessed the *FedNIDS* global model using the 20% testing set of adversarial samples, which was unique to each silo. In Stage 2, the *FedNIDS* model underwent a fine-tuning process using the additional 80% of adversarial training data, which was combined with the original training samples at each silo. Subsequently, we evaluated the proposed algorithm again on the same 20% testing set of adversarial samples.

The results, as illustrated in Figure 4, demonstrate that during Stage 1, the *FedNIDS* model encounters challenges in classifying adversarial samples, resulting in an average F1 score of 0.17 across all silos for both toolchains. This difficulty can be attributed to the inherent complexity of identifying adversarial evasion samples within the federated learning global model. However, after the fine-tuning process in Stage 2, *FedNIDS* demonstrates a noteworthy improvement in accuracy against adversarial samples. The average F1 score across all silos increases to 0.92. The fine-tuning mechanism enables the model to adapt and refine its decision boundaries based on the additional adversarial data, resulting in a significant improvement (around 75%) in the F1 score. This process enhances the model's ability to learn more robust features and decision rules, ultimately enabling it to better discriminate between benign and adversarial samples. These findings hold significant implications for real-world deployment scenarios where NIDS frequently faces adversarial attempts. These experiments underscore the effectiveness of our *FedNIDS* methodology in providing an adaptive and robust defense against novel network threats while maintaining data privacy and security.

6 CONCLUSIONS & FUTURE WORKS

In this study, we have presented the *FedNIDS*, a novel framework that leverages the power of federated learning and DNNs to enhance the accuracy, robustness, and privacy preservation of NIDS. Our framework addresses the challenge of accurate attack detection while respecting the privacy constraints of individual clients. The *FedNIDS* framework comprises two stages: federated DNN pre-training and federated novel attack fine-tuning. Through a comprehensive experimental evaluation, we demonstrated the effectiveness of *FedNIDS* in detecting specific attacks across different silos and its ability to adapt to new and emerging attack patterns. The quick adaptability of *FedNIDS* to changing attack patterns ensures that it remains effective even as new threats emerge. The robustness of our approach against adversarial attacks signifies its suitability for real-world scenarios where adversaries may attempt to evade detection.

Although our study has yielded promising results, below we discuss the assumptions in this study and practical considerations of deploying such a framework:

(1) *Reliable communication assumption.* In our experiments, we assume reliable communication channels between the silos and the central server. In real-world scenarios, factors like latency, bandwidth constraints, or communication costs could pose practical challenges.
(2) *Privacy and security assumption:* Our federated learning approach assumes a strong foundation of privacy and security to ensure that the learned weights transferred out from silos are secured. This is an underlying assumption in our approach and should be appropriately designed in real-world deployments.
(3) *Non-IID data in silos:* While our silo setup simulates a decentralized network, real-world conditions may introduce greater complexity in data distribution. Factors such as diverse hardware, software configurations, and network traffic patterns could intensify non-IID effects. Employing quantitative methods to assess

the non-IID nature of the data within silos, such as domain adaptation algorithms or advanced statistical analyses, would enhance the robustness of such an approach in a real-world setting.

(4) *Novelty detection and labeling:* Our work assumes the availability of novel attack samples during the fine-tuning stage. However, in the real world, security analysts at the affected silo are tasked with identifying novel attacks that bypass the local model. These newly labeled samples are then used for the fine-tuning stage to further update the global model.

There remain opportunities for future research. One avenue is examining alternative DNN architectures suitable for resource-constrained devices like IoT devices and sensors, which grapple with constraints in processing power, memory, storage capacity, and unreliable network connectivity. Furthermore, it would be valuable for the cybersecurity research community to investigate the delicate balance between accuracy and efficiency in these scenarios. Another intriguing avenue is exploring the integration of multi-modal data sources to achieve a more comprehensive understanding of network behaviors.

REFERENCES

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 308–318.

[2] Muhammad Ahmad, Qaiser Riaz, Muhammad Zeeshan, Hasan Tahir, Syed Ali Haider, and Muhammad Safeer Khan. 2021. Intrusion detection in internet of things using supervised machine learning based on application and transport layer features using UNSW-NB15 data-set. *EURASIP Journal on Wireless Communications and Networking* 2021, 1 (2021), 1–23.

[3] Mohiuddin Ahmed, Abdun Naser Mahmood, and Jiankun Hu. 2016. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications* 60 (2016), 19–31.

[4] Ammar Alazab, Ansam Khraisat, Sarabjot Singh, and Tony Jan. 2023. Enhancing Privacy-Preserving Intrusion Detection through Federated Learning. *Electronics* 12, 16 (2023), 3382.

[5] Ons Aouedi, Kandaraj Piamrat, Guillaume Muller, and Kamal Singh. 2022. FLUIDS: Federated Learning with semi-supervised approach for Intrusion Detection System. In *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 523–524.

[6] Manjula C Belavagi and Balachandra Muniyal. 2016. Performance evaluation of supervised machine learning algorithms for intrusion detection. *Procedia Computer Science* 89 (2016), 117–123.

[7] Aitor Belenguer, Jose A Pascual, and Javier Navaridas. 2023. GöwFed: A novel federated network intrusion detection system. *Journal of Network and Computer Applications* (2023), 103653.

[8] Vance W Berger and YanYan Zhou. 2014. Kolmogorov–smirnov test: Overview. *Wiley statsref: Statistics reference online* (2014).

[9] David A Bierbrauer, Michael J De Lucia, Krishna Reddy, Paul Maxwell, and Nathaniel D Bastian. 2023. Transfer learning for raw network traffic detection. *Expert Systems with Applications* 211 (2023), 118641.

[10] Philippe Biondi. 2010. Scapy documentation (!). *vol* 469 (2010), 155–203.

[11] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1175–1191.

[12] Jie Cui, Hu Sun, Hong Zhong, Jing Zhang, Lu Wei, Irina Bolodurina, and Debiao He. 2023. Collaborative Intrusion Detection System for SDVN: A Fairness Federated Deep Learning Approach. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[13] Michael J De Lucia, Paul E Maxwell, Nathaniel D Bastian, Ananthram Swami, Brian Jalaian, and Nandi Leslie. 2021. Machine learning raw network traffic detection. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications III*, Vol. 11746. SPIE, 185–194.

[14] Felix Erlacher and Falko Dressler. 2018. FIXIDS: A high-speed signature-based flow intrusion detection system. In *NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–8.

[15] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Maciá-Fernández, and Enrique Vázquez. 2009. Anomaly-based network intrusion detection: Techniques, systems and challenges. *computers & security* 28, 1-2 (2009), 18–28.

[16] Jalal Ghadermazi, Ankit Shah, and Nathaniel Bastian. 2023. Towards Real-time Network Intrusion Detection with Image-based Sequential Packets Representation. *TechRxiv preprint* (2023). https://doi.org/10.36227/techrxiv.23291588.v1

[17] John C Gower. 1971. A general coefficient of similarity and some of its properties. *Biometrics* (1971), 857–871.

[18] Ke He, Dan Dongseong Kim, and Muhammad Rizwan Asghar. 2023. Adversarial machine learning for network intrusion detection systems: a comprehensive survey. *IEEE Communications Surveys & Tutorials* (2023).

[19] Hanan Hindy, David Brosset, Ethan Bayne, Amar Kumar Seeam, Christos Tachtatzis, Robert Atkinson, and Xavier Bellekens. 2020. A taxonomy of network threats and the effect of current datasets on intrusion detection systems. *IEEE Access* 8 (2020), 104650–104675.

[20] Vanlalruata Hnamte and Jamal Hussain. 2023. Dependable intrusion detection system using deep convolutional neural network: A Novel framework and performance evaluation approach. *Telematics and Informatics Reports* 11 (2023), 100077. https://doi.org/10.1016/j.teler.2023.100077

[21] Soumyadeep Hore, Jalal Ghadermazi, Diwas Paudel, Ankit Shah, Tapas K Das, and Nathaniel D Bastian. 2023. Deep packgen: A deep reinforcement learning framework for adversarial network packet generation. *arXiv preprint arXiv:2305.11039* (2023).

[22] Soumyadeep Hore, Quoc Nguyen, Yulun Xu, Ankit Shah, Nathaniel Bastian, and Trung Le. 2023. Empirical Evaluation of Autoencoder Models for Anomaly Detection in Packet-based NIDS. In *Proceedings of the 2023 6th IEEE Conference on Dependable and Secure Computing*. IEEE.

[23] Meryem Janati Idrissi, Hamza Alami, Abdelkader El Mahdaouy, Abdellah El Mekki, Soufiane Oualil, Zakaria Yartaoui, and Ismail Berrada. 2023. Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems. *Expert Systems with Applications* 234 (2023), 121000.

[24] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning* 14, 1–2 (2021), 1–210.

[25] Min-Joo Kang and Je-Won Kang. 2016. Intrusion detection system using deep neural network for in-vehicle network security. *PloS one* 11, 6 (2016), e0155781.

[26] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. 2020. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 5132–5143. https://proceedings.mlr.press/v119/karimireddy20a.html

[27] Christopher Krügel, Thomas Toth, and Engin Kirda. 2002. Service specific anomaly detection for network intrusion detection. In *Proceedings of the 2002 ACM symposium on Applied computing*. 201–208.

[28] Vinod Kumar and Om Prakash Sangwan. 2012. Signature based intrusion detection system using SNORT. *International Journal of Computer Applications & Information Technology* 1, 3 (2012), 35–41.

[29] Beibei Li, Yuhao Wu, Jiarui Song, Rongxing Lu, Tao Li, and Liang Zhao. 2020. DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems. *IEEE Transactions on Industrial Informatics* 17, 8 (2020), 5615–5624.

[30] Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He. 2021. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering* (2021).

[31] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* 2 (2020), 429–450.

[32] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammdsadegh Saberian. 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing* 24, 3 (2020), 1999–2012.

[33] Patrick E McKnight and Julius Najab. 2010. Mann-Whitney U Test. *The Corsini encyclopedia of psychology* (2010), 1–1.

[34] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.

[35] Mohamed Amine Merzouk, Frédéric Cuppens, Nora Boulahia-Cuppens, and Reda Yaich. 2023. Parameterizing poisoning attacks in federated learning-based intrusion detection. In *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 1–8.

[36] Srinivas Mukkamala, Guadalupe Janoski, and Andrew Sung. 2002. Intrusion detection using neural networks and support vector machines. In *Proceedings of the 2002 International Joint Conference on Neural Networks. IJCNN'02 (Cat. No. 02CH37290)*, Vol. 2. IEEE, 1702–1707.

[37] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N Asokan, and Ahmad-Reza Sadeghi. 2019. DÏoT: A federated self-learning anomaly detection system for IoT. In *2019 IEEE 39th International conference on distributed computing systems (ICDCS)*. IEEE, 756–767.

[38] Guansong Pang, Chunhua Shen, and Anton van den Hengel. 2019. Deep anomaly detection with deviation networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 353–362.

[39] Sawsan Abdul Rahman, Hanine Tout, Chamseddine Talhi, and Azzam Mourad. 2020. Internet of things intrusion detection: Centralized, on-device, or federated learning? *IEEE Network* 34, 6 (2020), 310–317.

[40] Pedro Ruzafa-Alcázar, Pablo Fernández-Saura, Enrique Mármol-Campos, Aurora González-Vidal, José L Hernández-Ramos, Jorge Bernal-Bernabe, and Antonio F Skarmeta. 2021. Intrusion detection based on privacy-preserving federated learning for the industrial IoT. *IEEE Transactions on Industrial Informatics* 19, 2 (2021), 1145–1154.

[41] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. 2019. A detailed analysis of the cicids2017 data set. In *Information Systems Security and Privacy: 4th International Conference, ICISSP 2018, Funchal-Madeira, Portugal, January 22-24, 2018, Revised Selected Papers 4*. Springer, 172–188.

[42] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1 (2018), 108–116.
[43] Dug Song and Contributors. 2009. dpkt 1.9.2 documentation. https://dpkt.readthedocs.io/en/latest/. [Online; accessed 16-Feb-2023].
[44] K Muthamil Sudar and P Deepalakshmi. 2022. Flow-based detection and mitigation of low-rate ddos attack in sdn environment using machine learning techniques. In *IoT and Analytics for Sensor Networks: Proceedings of ICWSNUCA 2021*. Springer, 193–205.
[45] Pu Tian, Zheyi Chen, Wei Yu, and Weixian Liao. 2021. Towards asynchronous federated learning based threat detection: A DC-Adam approach. *Computers & Security* 108 (2021), 102344.
[46] Muhammad Fahad Umer, Muhammad Sher, and Yaxin Bi. 2017. Flow-based intrusion detection: Techniques and challenges. *Computers & Security* 70 (2017), 238–254.
[47] Nguyen Thanh Van, Tran Ngoc Thinh, et al. 2017. An anomaly-based network intrusion detection system using deep learning. In *2017 international conference on system science and engineering (ICSSE)*. IEEE, 210–214.
[48] Miel Verkerken, Laurens D'hooge, Tim Wauters, Bruno Volckaert, and Filip De Turck. 2022. Towards model generalization for intrusion detection: Unsupervised machine learning techniques. *Journal of Network and Systems Management* 30 (2022), 1–25.
[49] Bo Wang, Yang Su, Mingshu Zhang, and Junke Nie. 2020. A deep hierarchical network for packet-level malicious traffic detection. *IEEE Access* 8 (2020), 201728–201740.
[50] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated learning with matched averaging. *arXiv preprint arXiv:2002.06440* (2020).
[51] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H Vincent Poor. 2020. Tackling the objective inconsistency problem in heterogeneous federated optimization. *Advances in neural information processing systems* 33 (2020), 7611–7623.
[52] Mikal R Willeke, David A Bierbrauer, and Nathaniel D Bastian. 2023. Data-efficient, federated learning for raw network traffic detection. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications V*, Vol. 12538. SPIE, 247–262.
[53] Kai Yang, Yuanming Shi, Yong Zhou, Zhanpeng Yang, Liqun Fu, and Wei Chen. 2020. Federated machine learning for intelligent IoT via reconfigurable intelligent surface. *IEEE network* 34, 5 (2020), 16–22.
[54] Run Yang, Hui He, Yulong Wang, Yue Qu, and Weizhe Zhang. 2023. Dependable federated learning for IoT intrusion detection against poisoning attacks. *Computers & Security* 132 (2023), 103381.
[55] Binhang Yuan, Song Ge, and Wenhui Xing. 2020. A federated learning framework for healthcare iot devices. *arXiv preprint arXiv:2005.05083* (2020).
[56] Si-si Zhang, Jian-wei Liu, and Xin Zuo. 2021. Adaptive online incremental learning for evolving data streams. *Applied Soft Computing* 105 (2021), 107255.